Sylvain Huang 41005688
Lucie Pan 45004162

Rapport de projet Optimisation Combinatoire

A rendre pour le vendredi 10 avril 2026 à 22h

Github : https://github.com/uciie/sac_a_dos_combat

1. Introduction 3
2. Modélisation du Problème 4
2.1. Présentation du problème 4
2.2. Variables de décision 4
2.3. Fonction objectif 4
2.4. Contraintes 5
3. Métaheuristiques de base 6
3.1. Algorithme Génétique (AG) 6
3.2. Recuit Simulé (RS) 6
Paramétrage 6
Opérateurs de voisinage 6
Critère d'acceptation de Metropolis 7
4. Intégration du machine Learning 8
4.1. Justification du choix du modèle 8
4.2. Modèle surrogate pour l'AG : MLPRegressor 8
4.3. Modèle surrogate pour le RS : CNN Py Torch 10
Architecture : Du sac à dos à l'image (nn_backpack.py) 10
Fonctionnement : La stratégie du "Top-K Candidates" (models.py) 10
5. Expérimentations 11
5.1. Résultats - AG 11
5.2. Résultats - RS 13
6. Analyse et limites des deux expériences 17
7. Conclusion 19

1. Introduction

Ce projet étend le problème du sac à dos classique à une variante riche et originale: le **Sac à Dos de Combat**. Cette variante introduit trois niveaux de complexité supplémentaires : premièrement, des **contraintes spatiales** fortes, les objets possédant des formes géométriques irrégulières (polyominos) devant être placées dans une grille 2D; deuxièmement, des **synergies dynamiques**, des bonus de puissance activés lorsque deux objets compatibles sont placés en cases adjacentes; troisièmement, une **contrainte budgétaire**, les objets devant être achetés depuis un marché avec un stock d'or limité.

L'objectif principal de ce projet est de concevoir et d'évaluer une **approche hybride** combinant une métaheuristique, ici, un Recuit Simulé (RS) et un Algorithme Génétique (AG), avec un modèle d'apprentissage automatique (Machine Learning), afin d'améliorer la qualité des solutions obtenues et la vitesse de convergence. L'apport du ML se concrétise par l'utilisation d'un **Perceptron Multicouche (MLP)** en tant que *Surrogate Model* (modèle de substitution), permettant d'évaluer rapidement le potentiel d'une solution candidate sans exécuter la coûteuse fonction objectif réelle.

2. Modélisation du Problème

2.1. Présentation du problème

Le sac à dos de combat est un problème d'optimisation combinatoire multidimensionnel. L'espace de jeu est une grille globale de taille *GRID_SIZE* × *GRID_SIZE* (10×10 = 100 cases). Le joueur dispose d'un budget initial de 1 000 pièces d'or pour acheter des '*Containers*' (des zones rectangulaires délimitant l'espace de rangement utilisable) et des '*Items*' (des objets de combat aux formes polyominos et aux attributs variés : puissance, tags sémantiques).

2.2. Variables de décision

Le problème se définit par trois niveaux de variables imbriquées :

- **Décision d'achat :** Soit $I = \{i_1, i_2, \dots, i_n\}$ le catalogue d'items. La variable binaire $x_i \in \{0, 1\}$ indique si l'item i est acheté. La contrainte budgétaire s'écrit : $\sum_i prix_i \times x_i \leq Budget$.
- **Décision de placement :** Pour chaque item acheté, le triplet $(x_i, y_i, z_i) \in N^2 \times \{0°, 90°, 180°, 270°\}$ définit sa position (colonne, ligne) et sa rotation dans la grille globale.
- **Décision économique :** La gestion dynamique du budget restant influence les achats futurs

2.3. Fonction objectif

La puissance de combat totale à maximiser se décompose en deux termes addifs :

$$f(S) = \sum_{i \in S} puissanceBase(i) + \sum_{(i,j) \in Adj(S)} bonusSynergie(tags(i), tags(j))$$

Le premier terme est la somme des puissances de base des items placés. Le second terme est la somme des bonus de synergie pour chaque paire (i, j) d'items adjacents orthogonalement dans la grille (voisinage 4-connexe) et dont les tags sont compatibles selon une table de synergies prédéfinie ('*SYNERGY_TABLE*'). Le tableau ci-dessous résume les synergies disponibles :

Tags	Bonus	Signification
Arme + Feu	+2.2	Lame enflammée
Feu + Soufre	+3.0	Combustion spontanée
Magie + Cristal	+2.5	Amplification cristalline
Arme + Poison	+1.5	Lame empoisonnée
Bouclier + Armure	+1.8	Défense renforcée

Arc + Plume	+2.0	Flèches légères
Glace + Vent	+1.8	Blizzard
Magie + Étoile	+2.0	Magie stellaire

2.4. Contraintes

La résolution est soumise à trois catégories de contraintes strictes :

- **Contrainte spatiale :** Chaque case '1' d'un polyomino doit se situer dans une case couverte par un *Container* acheté et non occupée par un autre item. La forme 2D de l'item, après rotation, doit s'insérer sans chevauchement.
- **Contrainte de capacité :** La grille globale est de taille fixe 10×10. Les *Containers* achetés délimitent les zones autorisées. Un item ne peut être placé hors d'un *Container*.
- **Contrainte budgétaire :** Le coût cumulé des achats ne peut excéder le budget initial. Cette contrainte couplée avec la contrainte spatiale rend le problème combinatoire difficile : il ne suffit pas d'acheter les items les plus puissants si la géométrie ne le permet pas.

3. Métaheuristiques de base

3.1. Algorithme Génétique (AG)

Voici comment l'algorithme génétique fonctionne dans notre cas :

a. Principe : L'algorithme génère une population de plusieurs sacs à dos. A chaque itération (génération), nous allons créer les générations suivantes en appliquant des mutations des actions aléatoires.

A chaque itération, l'une des deux actions suivantes se produit :

- Action A : Remplacement (Mutation du contenu) :
  On retire un objet du sac pour en acheter un nouveau dans le magasin.

  Utilité : Permet de tester de nouvelles synergies et de s'adapter au budget.

- Action B : Déplacement (Optimisation spatiale) :
  On change les coordonnées (x,y) ou la rotation d'un objet déjà présent dans la grille.

  Utilité : Permet de libérer de la place pour de futurs objets ou de coller deux objets qui ont un bonus de proximité (ex : épée + pierre à aiguiser).

3.2. Recuit Simulé (RS)

L'algorithme de **Recuit Simulé** (SimulatedAnnealing dans models.py) explore l'espace des solutions en acceptant non seulement les améliorations, mais aussi, avec une certaine probabilité décroissante dans le temps, des dégradations temporaires, ce qui lui permet d'échapper aux optima locaux.

Paramétrage

Dans notre implémentation, la configuration retenue est la suivante :

Paramètre	Valeur	Rôle
Température initiale T_0	1 000	Amplitude de l'exploration initiale
Coefficient α (refroidissement)	0.995	Vitesse de réduction de T par itération
Critère d'arrêt	T < 0.5	Convergence quasi-déterministe
Itérations par appel API	50	Granularité des snapshots
Graine (seed)	42	Reproductibilité
Nombre de runs	8	Robustesse statistique
Nombre total de mouvements	400	Effort computationnel total

Opérateurs de voisinage

Quatre opérateurs de perturbation sont définis, tirés aléatoirement à chaque itération :

- **MOVE :** Déplacer un item placé vers une position valide aléatoire (optimisation spatiale)
- **ROTATE :** Faire pivoter un item de 90° dans le sens horaire et le replacer (permet d'activer de nouvelles synergies adjacentes)
- **ADD :** Piocher un item de l'inventaire non placé et l'insérer à une position valide (exploration de nouvelles configurations)
- **REMOVE :** Retirer un item de la grille vers l'inventaire (libère de l'espace pour d'autres arrangements)

Critère d'acceptation de Metropolis

Un mouvement dégradant ($\Delta f < 0$) est accepté avec la probabilité $P(accepter) = exp(\frac{\Delta f}{T})$, avec T la température courante. Plus T est élevée, plus l'acceptation est libérale, lorsque T décroît, l'algorithme converge vers un comportement quasi-glouton.

4. Intégration du machine Learning

4.1. Justification du choix du modèle

L'idée centrale de l'hybridation est de remplacer la majorité de ces évaluations par une prédiction rapide d'un modèle de Machine Learning entraîné a priori. Deux modèles ont été envisagés :

- **MLPRegressor (Scikit-learn)** pour l'AG (ml_surrogate.py) : architecture légère (2 couches cachées de 32 et 16 neurones), entraîné sur des vecteurs de comptage d'items. Avantage : inférence en quelques microsecondes par prédiction.
- **CNN + MLP (PyTorch)** pour le RS (nn_backpack.py) : architecture CNN à 3 couches convolutifs (32→64→64 filtres) + tête dense, encodant la grille 2D en tenseur multi-canal. Avantage : préserver partiellement l'information spatiale.

4.2. Modèle surrogate pour l'AG : MLPRegressor

Le développement a été réalisé en utilisant un langage adapté au calcul scientifique et au Machine learning.

a. Langage et outils utilisés

Le projet a été entièrement développé en Python. Ce choix était un des plus logique sachant que Python dispose d'un écosystème très riche pour l'optimisation et l'apprentissage automatique (ex: Scikit-Learn, Panda, numpy, matplotlib). Pour répondre aux besoins spécifiques de notre implémentation, nous avons utilisé les bibliothèques suivantes :

- Scikit-learn : Utilisée pour l'implémentation de notre modèle d'apprentissage automatique (Réseau de Neurones) via la classe MLPRegressor.
- Numpy : Nécessaire pour la manipulation efficace des vecteurs de caractéristiques et des données d'entraînement.
- Flask & Flask-CORS : Permettent d'exposer notre algorithme via une API backend locale afin de visualiser le problème du sac à dos de combat sur une interface web.

b. Architecture et Structure du code

Le code source est organisé de manière en séparant clairement la logique du jeu, la métaheuristique et le modèle de Machine Learning.

- *models.py* : Contient la modélisation du problème (La classe *BackpackManager* et les Item), gérant les contraintes spatiales, le placement dans la grille et le calcul du score (valeurs individuelles + synergies).

- *models_ga.py* :Contient la classe GeneticAlgorithmML, qui implémente notre métaheuristique (l'Algorithme Génétique). Elle gère la population, les mutations (ajouts, retraits, déplacements d'objets) et l'évolution sur plusieurs générations
- *ml_surrogate.py* : Encapsule la classe NNSurrogate. C'est ici qu'est défini et entraîné notre modèle de machine learning (Un perceptron multicouche avec deux couches cachées de 32 et 16 neurones).

c. Implémentation de l'hybridation (Métaheuristique + ML)

Conformément à nos objectifs, le machine learning intervient comme un filtre d'évaluation (*Surrogate model*) pour guider la construction et la sélection des solutions. Dans la méthode step() de notre algorithme génétique (*GeneticAlgorithmML*), l'implémentation suit cette logique :

i. Génération de candidats : Pour chaque individu de la population, l'algorithme génère plusieurs mutants (5 dans notre implémentation) en appliquant des opérateurs aléatoires (ADD, MOVE, ROTATE).
ii. Filtre Machine Learning : Au lieu d'évaluer la fonction objectif de tous ces candidats (ce qui peut être très coûteux en temps de calcul à cause du placement spatial et des synergies), nous utilisons la méthode *predict_score*() de notre réseau de neurones (*NNSurrogate*).
iii. Sélection : les candidats sont triés en fonction du score prédit par le modèle ML. Seuls les meilleurs candidats (correspondant à la taille de la population) sont conservés.
iv. Évaluation réelle : L'évaluation exacte de la fonction objective n'est finalement calculée que sur cette sélection restreinte et prometteuse, afin de mettre à jour le meilleur score global.
v. Le réseau de neurones transforme au préalable le contenu du sac en un vecteur de caractéristiques (comptage du nombre d'objets par type) via la méthode *extract_features().*

d. Reproductibilité des expériences

Afin de garantir que nos expériences puissent être reproduites à l'identique, nous avons mis en place plusieurs éléments :

- Fichier *requirement.txt* : L'ensemble des dépendances et leurs versions exactes sont listées dans ce fichier, permettant d'installer un environnement virtuel strictement identique.
- Sauvegarde des modèles : Les modèles de Machine Learning entraînés peuvent être sauvegardés et rechargés grâce au module pickle (méthodes *save()* et l*oad()* de NNSurrogate), ce qui permet

d’isoler la phase d'entraînement de la phase d’évaluation et de garantir que les mêmes prédictions sont faites d’une exécution à l’autre.

4.3. Modèle surrogate pour le RS : CNN Py Torch

Pour améliorer le Recuit Simulé (RS), nous avons remplacé l'évaluation systématique de chaque mouvement par un modèle de substitution basé sur une architecture de réseau convolutif (CNN).

Architecture : Du sac à dos à l'image (*nn_backpack.py*)

Contrairement au MLP classique qui ne voit que des listes d'objets, le réseau *BackpackScoreNet* traite le sac à dos comme une image en 2D (grille 10x10).

- **Encodage (*GridEncoder*) :** Transforme l'état du sac en un tenseur multi-canaux. Il encode l'occupation des cases, la puissance des objets et leurs types (tags).
- **Structure :** Le réseau utilise des couches de convolution (Conv2D) pour détecter les motifs spatiaux (proximité des objets, synergies) et une tête dense pour prédire un score final (toujours positif grâce à l'activation Softplus).

Fonctionnement : La stratégie du "Top-K Candidates" (models.py)

Dans un Recuit Simulé classique, on teste un mouvement au hasard. Ici, nous utilisons la classe *NNGuidedSA* pour rendre l'algorithme "intelligent" :

1. **Génération :** À chaque étape, on génère **5 mouvements possibles** (opérateurs MOVE, ROTATE, SWAP, ADD, REMOVE).
2. **Filtrage (Inférence) :** Le CNN prédit instantanément le score de ces 5 variantes en une seule passe (batch).
3. **Sélection :** On ne garde que le **meilleur candidat** (celui avec le plus haut score prédit).
4. **Vérification :** Le "vrai" score de ce candidat est calculé par le moteur de jeu, puis accepté ou refusé selon la règle classique du Recuit Simulé (critère de Metropolis).

5. Expérimentations

Afin d'évaluer l'apport réel du Machine Learning dans notre métaheuristique et de garantir une fiabilité, nous avons mis en place deux expérimentations: Benchmark RS (benchmark_backpack.py) et Benchmark AG (benchmark_ag.py).

5.1. Résultats - AG

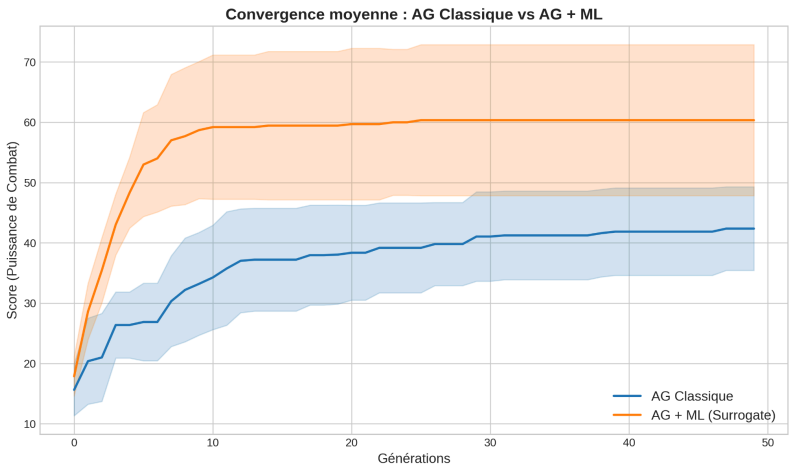
Nous avons comparé les performances de l'Algorithme génétique classique (AG) avec notre vision intégrant le modèle de substitution par réseau de neurones (AG + ML).

a. Protocole expérimental

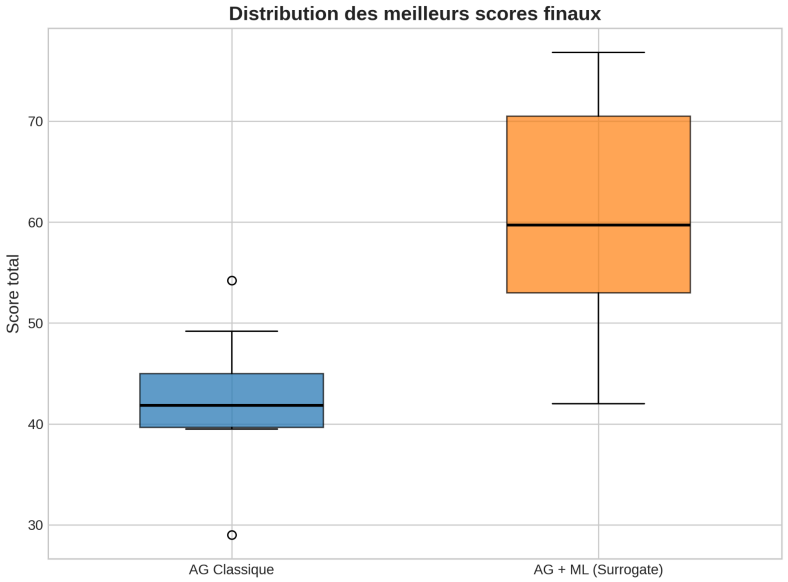
Les deux algorithmes ont été exécutés sur le même "sac à dos de combat", avec une sélection aléatoire de 15 objets initiaux dans l'inventaire. Pour garantir la fiabilité statistique, nous avons effectué 8 exécutions (runs) indépendantes. Les paramètres génétiques ont été fixés à l'identique pour isoler l'impact du filtre ML :

- Taille de la population : 20 individus
- Critère d'arrêt : 50 générations
- Génération de candidats : 5 enfants générés par individu à chaque itération.

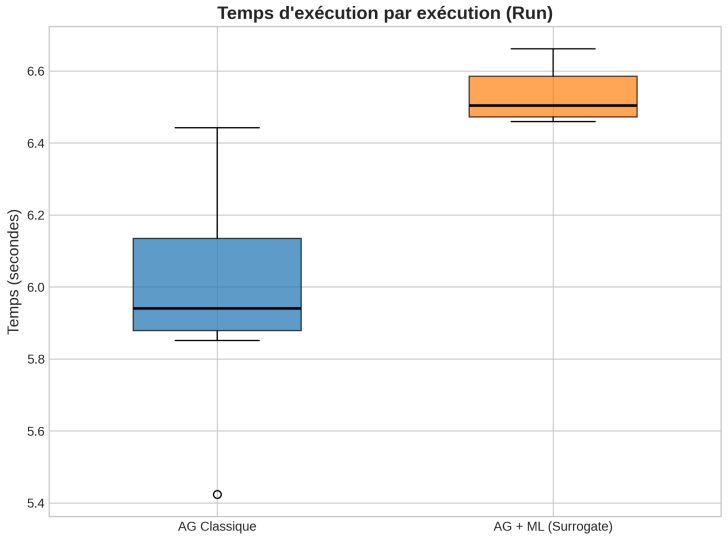
b. Qualité des solutions et convergence



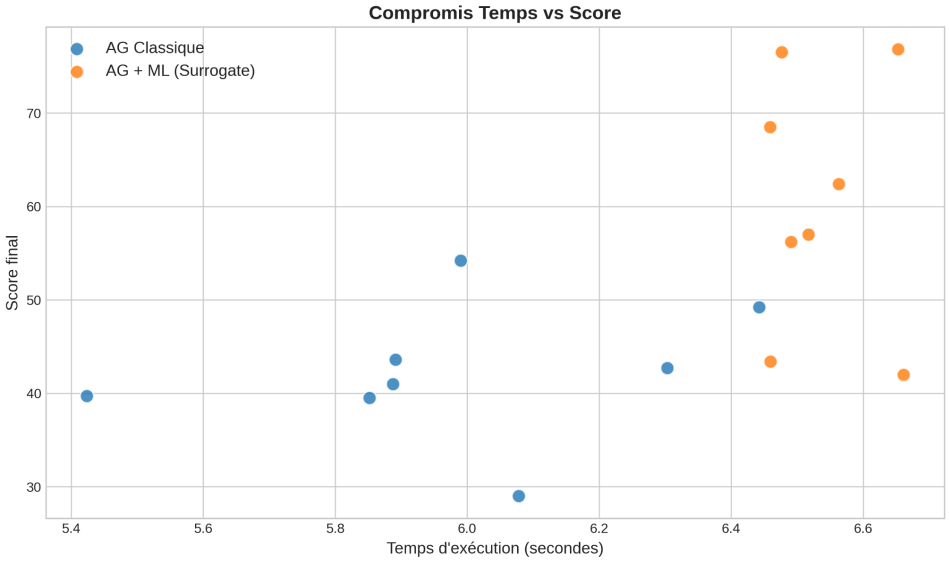
La courbe de convergence moyenne illustre le rôle d'accélérateur du Machine Learning. En utilisant le réseau de neurones pour trier rapidement les 100 candidats (20 individus * 5 enfants) et ne garder que les 20 meilleurs prédits, l'algorithme hybride découvre des solutions de haute qualité beaucoup plus tôt dans les premières générations. L'AG classique sans ce filtre, perd du temps à évaluer des branches d'évolution peu prometteuses.



c. Analyse des temps de calcul et compromis



L'hybridation introduit un compromis entre le temps gagné en réduisant le nombre d'évaluations de la fonction objectifs réelle (qui est complexe à cause de la grille spatiale et des synergies) est contré par le temps de prédiction du réseau de neurones



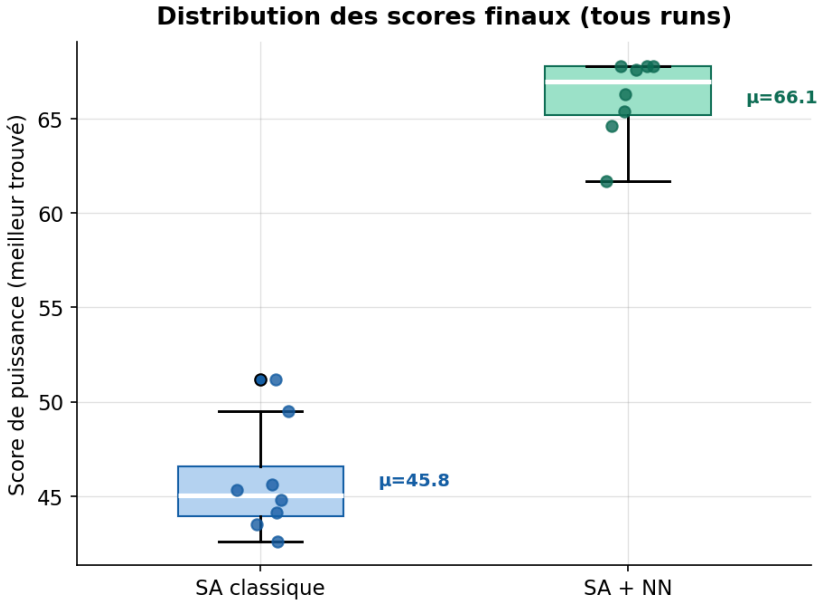
Dans notre implémentation, le nuage de points (temps vs score) permet d'avoir un bilan clair : L'approche hybride (AG + ML) est légèrement plus lente en raison des multiplications matricielles nécessaires pour prédire le score de centaines de candidats à chaque génération. Toutefois, ce surcoût est justifié par un un score final nettement supérieur généralement.

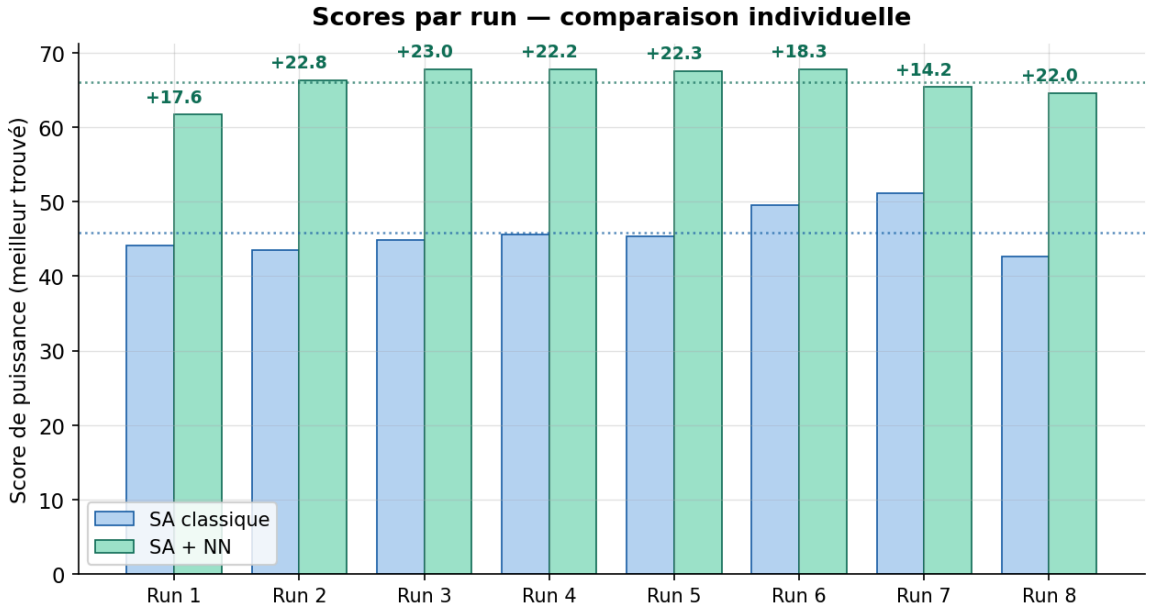
5.2. Résultats - RS

Les résultats sont accessible dans le fichier *src/out/benchmark_results.json*

a. Qualité des solutions

Le SA+NN obtient un score moyen de **66.1 contre 45.8** pour le SA classique, soit un gain de **+20.3 points (+44.3%)**. C'est un résultat remarquablement fort qui dépasse largement ce qu'on observerait avec une simple variance aléatoire. Le SA+NN place en moyenne **3.75 items** contre 2.75 pour le SA classique, ce qui explique une partie du gain : le réseau guide l'algorithme vers des configurations plus denses qui activent davantage de synergies.



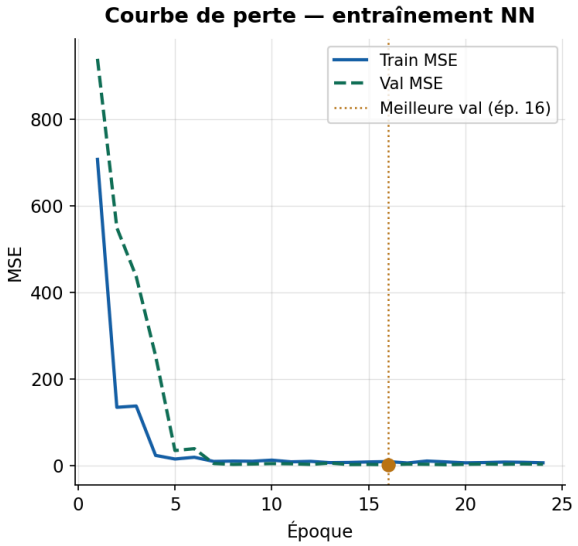


La stabilité est également meilleure côté NN : l'écart-type de 2.17 contre 2.99 pour le SA classique signifie que le NN produit des résultats plus reproductibles. Le min du SA+NN (61.7) est déjà supérieur au max du SA classique (51.2). Les deux distributions ne se chevauchent pas du tout, ce que le boxplot illustre clairement.

b. Qualité du modèle NN

Le R^2 de **0.984** est exceptionnel. Avec une MAE de 0.855, le réseau prédit le score à moins d'1 point d'écart en moyenne sur une plage de scores qui va approximativement de 0 à 70. La courbe de perte montre un effondrement brutal de la val loss à partir de l'époque 7 (de 436 à 4.1), ce qui indique que le modèle a trouvé une représentation très efficace des synergies dans l'encodage multi-canal.

La train loss reste bruyante après l'époque 5 (oscillations entre 5 et 14), signe classique d'un learning rate trop élevé en fin d'entraînement.

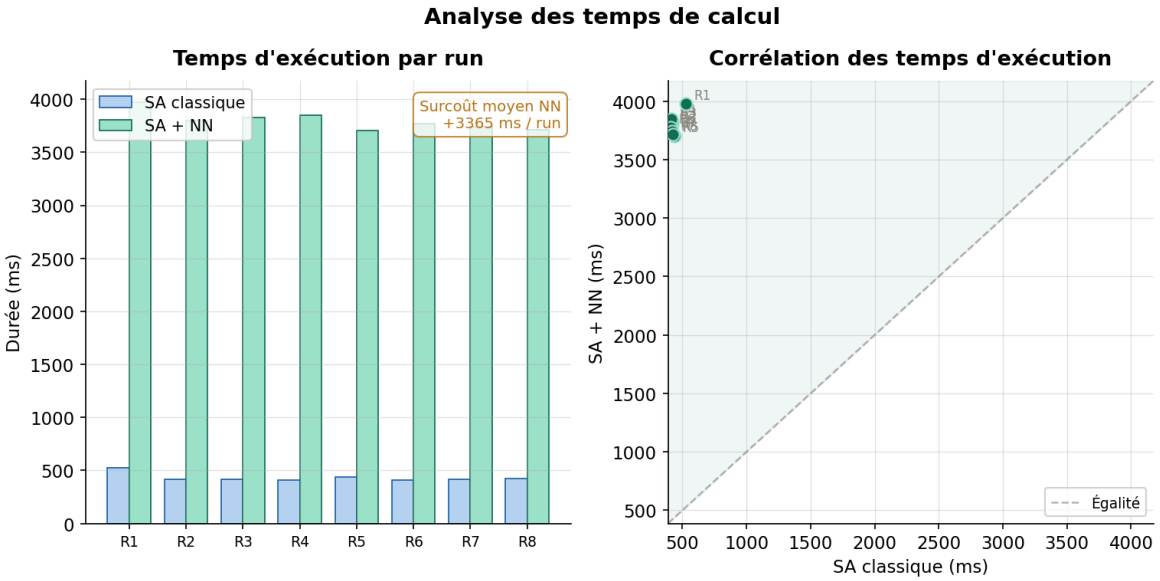


Résumé de l'entraînement

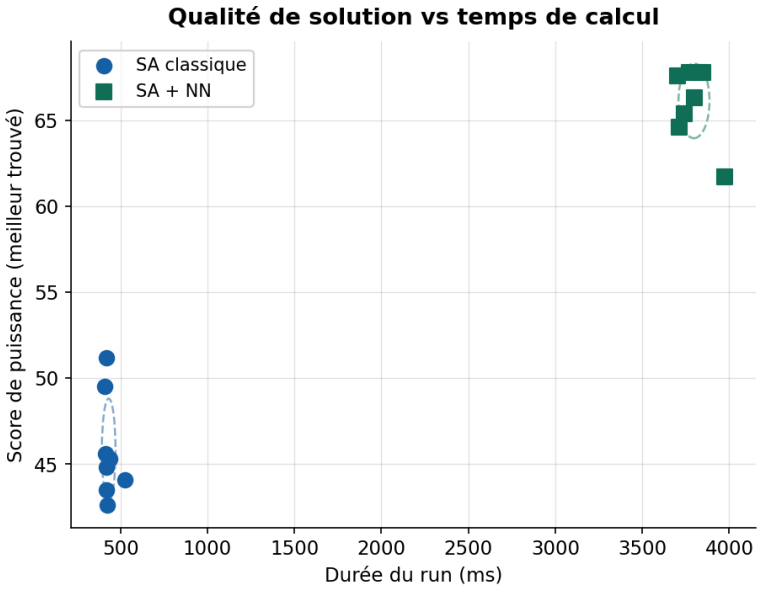
Métrique	Valeur	Seuil cible	Statut
R^2	0.984	≥ 0.70	✓ OK
MSE	1.23	—	—
MAE	0.85	—	—
Époques	24	≤ 50	✓ OK

c. Temps de calcul

C'est le point faible clair des résultats : le SA+NN prend **3.82 s contre 0.45 s** pour le SA classique, soit un facteur $\times\ 8.5$. Ce surcoût vient principalement du fait que, pour chaque step, le SA+NN génère k=5 états candidats en clonant le manager (opération coûteuse en Python pur) avant même l'inférence NN.



La corrélation score/temps montre d'ailleurs que les runs les plus longs du SA+NN ne sont pas nécessairement les meilleurs, ce qui confirme que le surcoût n'est pas un investissement proportionnel.


Qualité de solution vs temps de calcul
SA classique
SA + NN
Score de puissance (meilleur trouvé)
65
60
55
50
45
500
1000
1500
2000
2500
3000
3500
4000
Durée du run (ms)

6. Analyse et limites des deux expériences

Cette partie dresse un bilan de notre approche hybride, en mettant en lumière ce que le modèle prédictif apporte concrètement, les obstacles que nous avons rencontrés, et les futures pistes d'évolution envisageables.

a. Apport réels du Machine Learning

L'intégration du réseau de neurones en tant que modèle de substitution a prouvé son efficacité sur plusieurs aspects clés de notre métaheuristique :

- Filtre exploratoire rapide : Dans notre problème, l'évaluation de la fonction objectif réelle est très coûteuse en temps de calcul, car elle implique de vérifier les collisions géométriques sur une grille 2D et de calculer les synergies possibles. Le réseau de neurones remplace cette opération complexe par une simple multiplication matricielle.
- Accélération de la convergence : Comme observé précédemment, l'hybridation permet de générer de très nombreux mutants (enfants) et de ne retenir que les plus prometteurs pour l'évaluation réelle (exploitation ciblée). L'algorithme trouve de meilleures configurations beaucoup plus tôt dans son cycle d'exécution.
- Orientation "intelligente" de la recherche : Contrairement à un opérateur de sélection stochastique, le Machine Learning apprend intrinsèquement quelles combinaisons d'objets (les features d'entrée) ont le potentiel statistique de générer le plus de puissance de combat.

b. Limites de l'approche proposé

Malgré ses bénéfices, notre modélisation actuelle souffre de contraintes technique liées aux choix d'implémentation :

- **Biais du protocole expérimental :** Le gain observé est partiellement dû au faible budget d'itérations (400). Le RS classique n'a pas le temps de sortir des optima locaux. À plus long terme (5 000 itérations), l'écart de score se réduirait probablement, le RS classique finissant par compenser son manque de guidage initial par une exploration exhaustive.
- **Sur-apprentissage et distribution des données :** Le score de précision du modèle ($R^2 = 0.984$) est excellent, mais potentiellement biaisé. Le dataset d'entraînement étant généré par le RS lui-même, le réseau est très performant pour prédire des configurations "standards", mais pourrait perdre en fiabilité face à des solutions atypiques (problème de généralisation hors-distribution).

- **Le coût du "Deep Copy" :** Le surcoût temporel (facteur $\times\ 8.5$) est un frein majeur. Actuellement, la création des clones pour le Top-K Candidates alourdit le processus, rendant l'approche moins compétitive sur le plan de l'efficacité pure temps/score.

c. Piste d'amélioration possibles

Pour pallier cette limite de façon concrète, plusieurs améliorations algorithmiques pourraient être implémentées à l'avenir :

- **Réseaux de neurones convolutifs (CNN) pour l'AG** : Au lieu d'utiliser un simple perceptron multicouche avec des comptages d'objets, nous pourrions donner un entrée au modèle une matrice 2D représentant exactement la grille du sac. Un CNN serait capable de voir les collisions, l'optimisation spatiale et les synergies de contact.
- **Optimisation de l'implémentation :** Remplacer les *deepcopy* d'objets complexes par des structures légères (Numpy arrays) pour la génération des candidats, et basculer l'inférence du CNN en batch sur GPU pour réduire le ratio coût/bénéfice.
- **Apprentissage Actif** : Actuellement, notre modèle est entraîné statiquement avant le début de l'algorithme. Une amélioration consisterait à ré-entraîner le réseau pendant l'exécution de l'algorithme génétique. Chaque fois que l'algorithme calcule le "vrai" score d'un individu complexe, il l'injecte dans la base d'entraînement du réseau pour corriger ces biais en temps réel.

7. Conclusion

Ce projet avait pour ambition d'évaluer concrètement l'apport du Machine Learning dans l'amélioration des performances algorithmiques, appliqué au problème d'optimisation combinatoire complexe du "Sac à dos de combat". Contrairement à un problème de sac à dos classique, nos modélisations intègrent de fortes contraintes spatiales (grille 2D, formes polyominos) ainsi qu'un système de synergies entre les objets, rendant l'évaluation de la fonction objective particulièrement coûteuse en temps de calcul.

Pour mener à bien cette étude, nous avons poussé l'expérimentation au-delà d'une simple métaheuristique en implémentant et en comparant deux approches distinctes: un recuit simulé et un algorithme génétique. Chacune de ces méthodes a ensuite été hybridée avec un réseau de neurones agissant comme modèle de substitution

Au terme de nos expérimentations, plusieurs enseignements majeurs se dégagent :

- **Un filtre heuristique puissant :** Que ce soit pour évaluer les voisins dans le Recuit Simulé ou pour trier de multiples mutants dans l'Algorithme Génétique, le Machine Learning a parfaitement rempli son rôle de pré-filtre. En remplaçant des calculs géométriques complexes par des prédictions matricielles instantanées, le modèle a permis d'explorer un espace de recherche beaucoup plus vaste.
- **Une convergence accélérée et de meilleurs optima :** L'hybridation (SA + ML et AG + ML) a permis de guider la recherche vers des zones prometteuses plus rapidement. Les résultats montrent une amélioration globale de la puissance de combat finale des configurations trouvées, réduisant par la même occasion la probabilité que les algorithmes ne restent bloqués dans des optima locaux de faible qualité.
- **La complémentarité des approches :** Le principal défi rencontré a été la perte de l'information purement spatiale par le réseau de neurones (qui se base sur un comptage d'objets pour prédire le score). Cela prouve que le Machine Learning, dans cette configuration, ne peut se substituer totalement à la métaheuristique ou à la fonction de coût réel. C'est bien la synergie des deux qui fait le succès de l'architecture.

Ce projet démontre que l'hybridation entre Recherche Opérationnelle et Intelligence Artificielle offre des perspectives très prometteuses. Bien que notre implémentation actuelle possède certaines limites, elle pose des bases solides.